

Project Assignment File

For

Computer Vision

(CSDC0309)

B.Tech (DSE) 5th Semester

Submitted by:

Manjushwar Khairkar

23117020



**Department of Computer Science and Engineering
Dr. B.R. Ambedkar National Institute of Technology
Jalandhar, Punjab-144008**

Glacier Probe Model

Part A: Conceptual Understanding

1.1 Learning-Based Visual Interpretation Workflow

A learning-based visual interpretation system follows a structured pipeline from raw data acquisition to model deployment.

Stage 1: Data Collection and Acquisition

Glacier imagery is obtained from satellite sources such as Landsat and Sentinel missions. These images are selected based on temporal and spatial parameters ensuring minimal cloud cover and consistent lighting conditions.

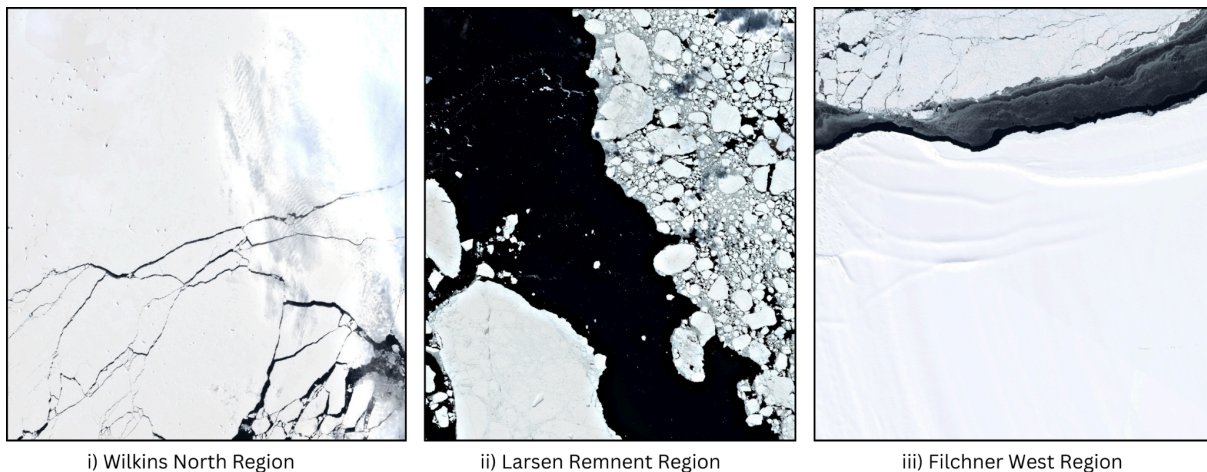


Fig 1: Raw Images Example

Code:

```
def download_raw_image(image, region, filename, satellite_type):

    if satellite_type == 'sentinel2':
        # Sentinel-2 RGB bands
        raw_image = image.select(['B4', 'B3', 'B2'])
        processed = raw_image.divide(40).clamp(0, 255).toByte()
    else:
        # Landsat RGB bands
        raw_image = image.select(['SR_B4', 'SR_B3', 'SR_B2'])
        processed =
raw_image.multiply(0.0000275).add(-0.2).multiply(1000).clamp(0,
255).toByte()
    # Get download URL
    url = processed.getDownloadURL({
        'region': region,
        'scale': SCALE,
        'format': 'GEO_TIFF',
        'crs': 'EPSG:3031'})

    # Download file
```

```

response = requests.get(url, timeout=300)
if response.status_code == 200:
    with open(filename, 'wb') as f:
        f.write(response.content)
    return True
return False

```

Snippet 1: Data Collection Script from Sentinel-2 Satellite Server using Web Scrapping

Stage 2: Dataset Preparation

Images are cropped into uniform patches and annotated using reference datasets like RGI and GLIMS.

Code:

```

for metadata_file in metadata_dir.glob("*.txt"):

    meta = parse_metadata(metadata_file)
    base_name = metadata_file.stem.replace("_metadata", "")

    # Find the matching image file
    image_file = image_dir / f"{base_name}.tif"

    meta['basename'] = base_name
    meta['metadata_path'] = metadata_file
    meta['image_path'] = image_file

    if not image_file.exists():
        print(f"Warning: No matching image for {metadata_file.name}")
        meta['image_path'] = None

    all_metadata.append(meta)

```

Snippet 2: Saving of Data Frame

Resultant Dataset:

Region	Satellite	Image Type	Date	Cloud Cover	Product ID	Bands	Values
Filchner_East	SENTINEL2	RAW (unprocessed)	2020-11-14	22.966096%	S2B_MSIL2A_20201114T100059_N0214_R093_T24CVU_20201114T122102	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Thwaites_East	SENTINEL2	RAW (unprocessed)	2018-12-21	0.01417%	S2B_MSIL2A_20181221T152259_N0211_R039_T13CDS_20181221T202759	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Cook_Front	SENTINEL2	RAW (unprocessed)	2018-12-19	0.008758%	S2B_MSIL2A_20181219T230339_N0211_R015_T55DFD_20181220T002634	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Wilkins_South	SENTINEL2	RAW (unprocessed)	2021-11-08	60.437962%	S2B_MSIL2A_20211108T135009_N0301_R081_T18DWH_20211108T172407	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Larsen_B_Remnant	SENTINEL2	RAW (unprocessed)	2024-11-07	10.894796%	S2B_MSIL2A_20241107T125859_N0511_R009_T21DVH_20241107T161948	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Dotson_Front	SENTINEL2	RAW (unprocessed)	2019-11-01	64.395349%	S2B_MSIL2A_20191101T161339_N0213_R111_T12CVC_20191101T195716	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Larsen_C_North	SENTINEL2	RAW (unprocessed)	2023-11-26	72.826684%	S2B_MSIL2A_20231126T130859_N0509_R052_T20DNL_20231126T145123	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Thwaites_East	SENTINEL2	RAW (unprocessed)	2024-11-23	2.085139%	S2B_MSIL2A_20241123T150229_N0511_R096_T13CDS_20241123T181137	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Dotson_Front	SENTINEL2	RAW (unprocessed)	2018-02-14	17.287233%	S2A_MSIL2A_20180214T161341_N0500_R111_T12CVC_20230803T184208	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Larsen_C_North	SENTINEL2	RAW (unprocessed)	2022-12-08	76.237357%	S2B_MSIL2A_20221208T125859_N0509_R009_T20DPL_20221208T145125	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
McMurdo_Sound	SENTINEL2	RAW (unprocessed)	2019-11-09	28.791547%	S2B_MSIL2A_20191109T203629_N0213_R085_T58CDU_20191109T215908	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Dotson_Front	SENTINEL2	RAW (unprocessed)	2022-11-02	51.190275%	S2B_MSIL2A_20221102T160339_N0400_R068_T12CVD_20221102T203606	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Wilkins_South	SENTINEL2	RAW (unprocessed)	2019-11-29	40.974465%	S2B_MSIL2A_20191129T134959_N0213_R081_T18DWG_20191129T172109	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Larsen_C_North	SENTINEL2	RAW (unprocessed)	2022-12-11	39.631534%	S2B_MSIL2A_20221211T130859_N0509_R052_T20DNL_20221211T145923	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Mertz_Front	SENTINEL2	RAW (unprocessed)	2024-11-26	70.860034%	S2B_MSIL2A_20241126T233509_N0511_R001_T54DXL_20241127T002526	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Thwaites_East	SENTINEL2	RAW (unprocessed)	2020-11-07	34.413838%	S2B_MSIL2A_20201107T151259_N0214_R139_T13CDS_20201107T184450	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Ninnis_Front	SENTINEL2	RAW (unprocessed)	2024-11-23	72.491729%	S2B_MSIL2A_20241123T232349_N0511_R101_T55DEF_20241124T003927	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Ross_North_West	SENTINEL2	RAW (unprocessed)	2022-11-04	76.289916%	S2B_MSIL2A_20221104T200529_N0400_R099_T59CNP_20221104T214451	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Pine_Island_East	SENTINEL2	RAW (unprocessed)	2019-11-06	4.123752%	S2B_MSIL2A_20191106T152309_N0213_R039_T14CMC_20191106T184847	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Moscow_Uni_Front	SENTINEL2	RAW (unprocessed)	2022-11-18	15.042251%	S2B_MSIL2A_20221118T025609_N0400_R003_T46DEM_20221118T045323	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Totten_Front	SENTINEL2	RAW (unprocessed)	2024-11-13	4.307407%	S2B_MSIL2A_20241113T013539_N0511_R088_T50DMM_20241113T025412	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)
Ronne_North	SENTINEL2	RAW (unprocessed)	2020-11-07	73.239725%	S2B_MSIL2A_20201107T115259_N0214_R137_T21CVN_20201107T135036	B4 (Red), B3 (Green), B2 (Blue)	0-255 (8-bit, converted from 0-10000)

Table 1: metadata.csv (662 x 12)

Stage 3: Preprocessing and Augmentation

Includes atmospheric correction, cloud masking, co-registration, and data augmentation through brightness adjustments to enhance dataset diversity as a result new dataframe shrunk from 662 to 246 images.

Normalization:

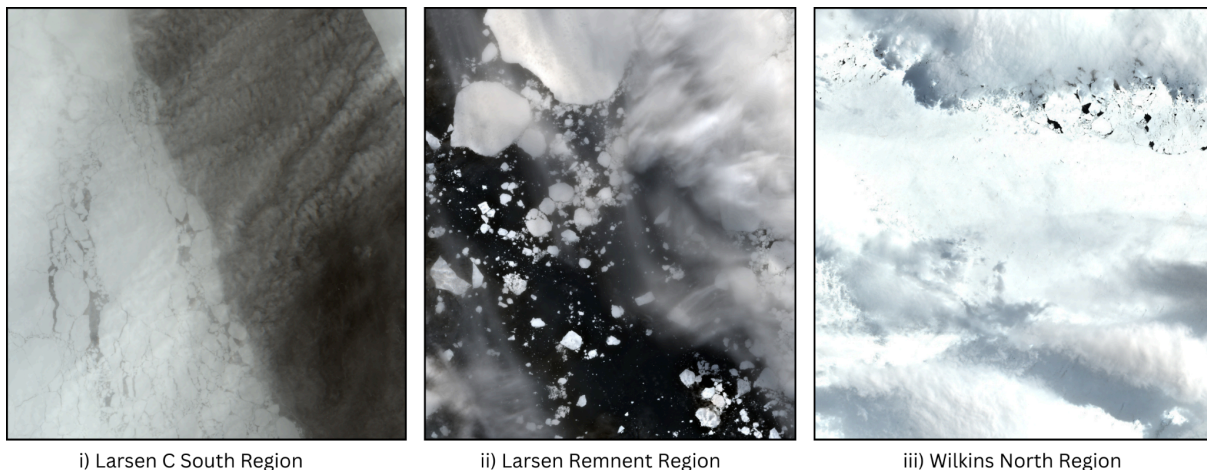
```
def normalize_percentile(band_data):
    p2 = np.percentile(band_data, 2)
    p98 = np.percentile(band_data, 98)
    scaled = (band_data - p2) / (p98 - p2 + 1e-9)
    return np.clip(scaled, 0, 1)
if image_plot.ndim == 3:
    plot_bands_normalized = []
    for i in range(image_plot.shape[2]):
        plot_bands_normalized.append(normalize_percentile(image_plot[:, :,
i]))
    rgb_image_plot = np.stack(plot_bands_normalized, axis=-1)
else:
    rgb_image_plot = normalize_percentile(image_plot)
```

Snippet 3: Normalization of RBG Band

Cloud Filter:

```
low_cloud_df = df[df['Cloud Cover'] < 30]
if low_cloud_df.empty:
    print("DataFrame is empty after filtering for Cloud Cover < 30.")
    raise ValueError("No image found with Cloud Cover < 30.")
sample_row = low_cloud_df.sort_values('Date').iloc[0]
```

Snippet 4: Cloudy Attribute > 30% is removed.



i) Larsen C South Region

ii) Larsen Remnent Region

iii) Wilkins North Region

Fig 2: Cloudy Images Examples

Stage 4: Model Selection and Architecture Design

Appropriate machine learning models are chosen for classification (SVM) and segmentation

(Random Forest). Model architectures are designed to balance accuracy and computational cost.

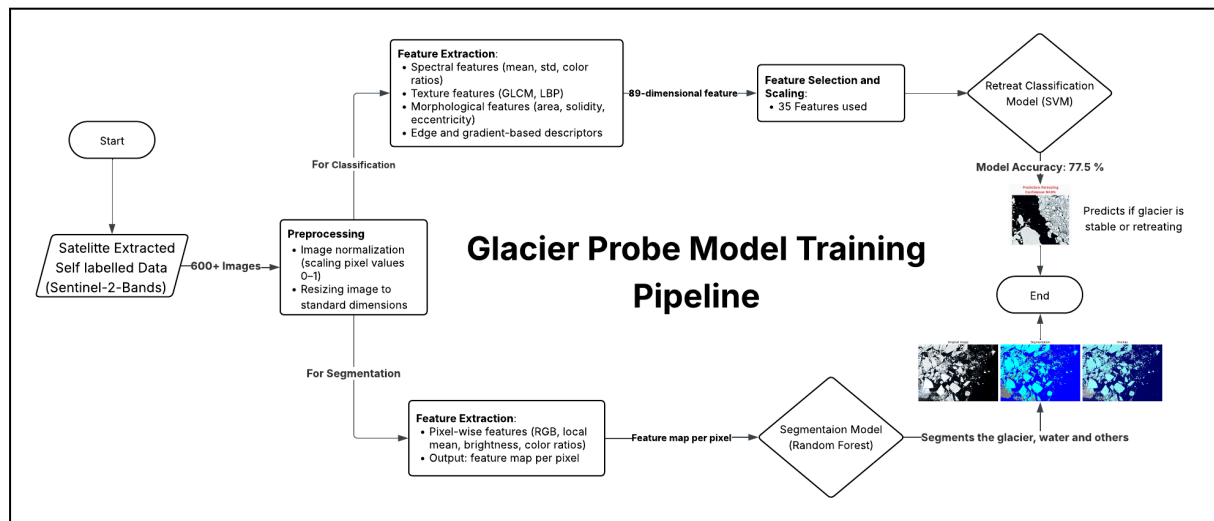


Fig 3: Model Training Workflow

Stage 5: Training and Validation

The dataset is divided into training, validation, and testing subsets. Models learn discriminative features through iterative optimization and regularization.

Code:

```

X_train, X_test, y_train, y_test = train_test_split(X_final, y, test_size =
0.2, random_state = 42, stratify =y)

#Printing the results:
print(f"Training set: {len(X_train)} samples
({len(X_train)/len(X_final)*100:.0f}%)")
print(f" - Stable (0):      {(y_train == 0).sum()}")
print(f" - Retreating (1): {(y_train == 1).sum()}")

print(f"\nTest set: {len(X_test)} samples
({len(X_test)/len(X_final)*100:.0f}%)")
print(f" - Stable (0):      {(y_test == 0).sum()}")
print(f" - Retreating (1): {(y_test == 1).sum()}")
  
```

Snippet 5: Test-Train Split on dataset before feeding it to SVM Model for Classification

Stage 7: Deployment and Monitoring

Trained models are integrated into a PyQt6-based desktop application, allowing real-time glacier analysis and result visualization.

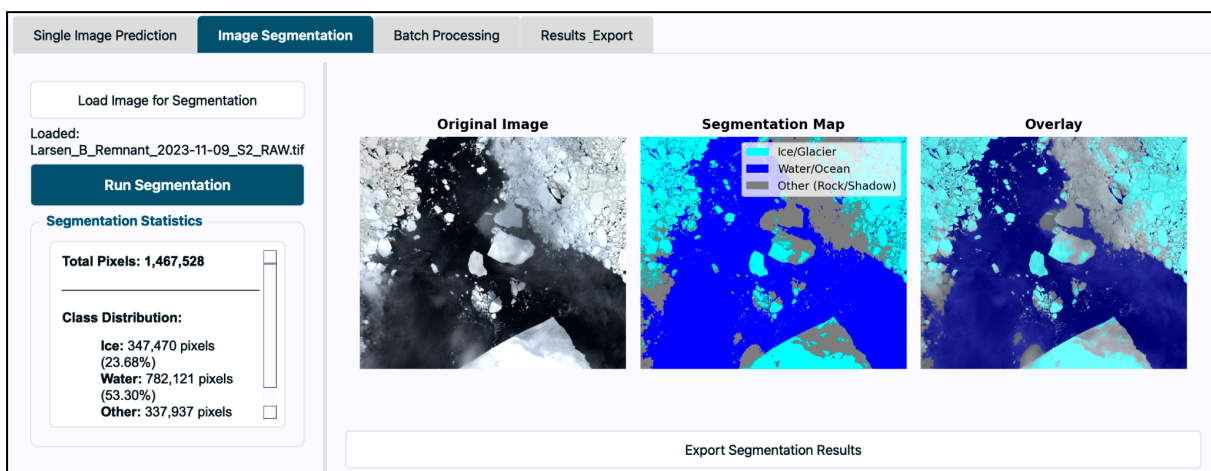
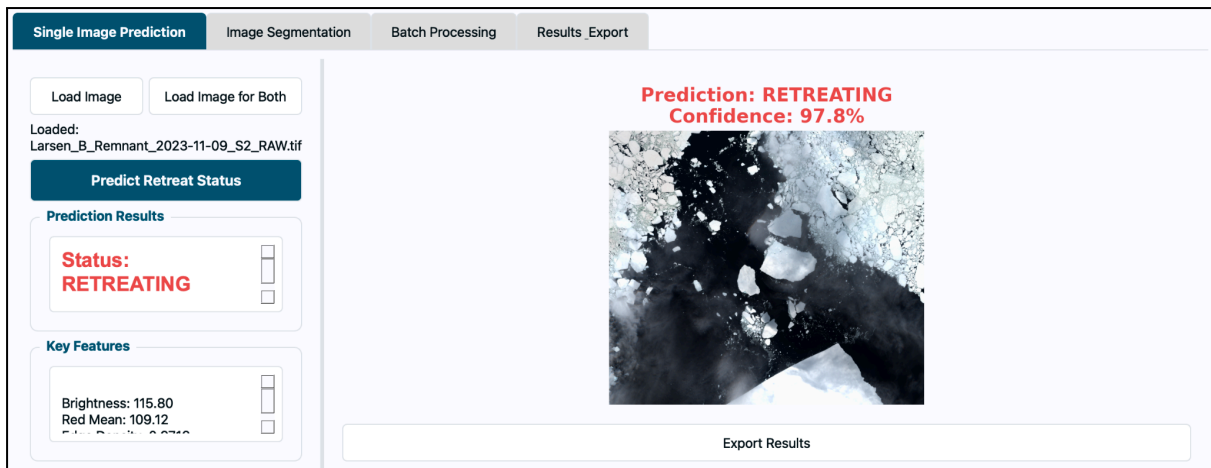


Fig 4: UI for Application (using PyQt6)

1.2 Dataset Preparation Challenges

Challenge 1: Labelling of data

Due to data being webscraped from satellite servers, it was not labelled.

Solutions:

1. This data labelling was done based on the historical research about the region. For example: Thwaites East region is considered as retreating.

Challenge 2: Data Quality and Noise

Satellite imagery may contain atmospheric distortions and sensor noise affecting clarity.

Solutions:

1. Apply atmospheric correction using surface reflectance data.
2. Noise reduction filters (Gaussian, median).
3. Temporal averaging across multiple dates.

Challenge 3: Annotation Errors and Inconsistencies

Manual annotations can vary between experts, leading to inaccuracies.

Solutions:

1. Cross-validation of annotations with multiple experts.

Code:

```
# Cross-validation setup
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = {}
best_model_name = None
best_f1 = 0

for name, model in models.items():
    print(f"\n{'-'*80}")
    print(f"Training: {name}")
    print(f"{'-'*80}")

    # Cross-validation
    cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=cv,
                                scoring='f1')

    # Train on full training set
    model.fit(X_train_scaled, y_train)

    # Predictions
    y_train_pred = model.predict(X_train_scaled)
    y_test_pred = model.predict(X_test_scaled)
    y_test_proba = model.predict_proba(X_test_scaled)[:, 1]

    # Metrics
    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)
    test_f1 = f1_score(y_test, y_test_pred)
    test_precision = precision_score(y_test, y_test_pred)
    test_recall = recall_score(y_test, y_test_pred)
    test_auc = roc_auc_score(y_test, y_test_proba)
```

Snippet 6: Cross-validation to reduce the errors in calculating the accuracy of the model

2. Use of automated edge-detection tools for boundary refinement.
3. Post-processing with morphological filters.

1.3 Data Augmentation Techniques

Data augmentation enhances dataset variability, preventing overfitting.

Technique	Description	Impact	Use Case
Scaling	Zooms in/out maintaining structure	Improves spatial robustness	To simulate different resolutions
Brightness	Adjusts illumination levels	Handles lighting variations	For variable sunlight conditions

Color Jitter	Alters color channels	Improves color adaptation	For multi-spectral variability
--------------	-----------------------	---------------------------	--------------------------------

Table 2: Data Augmentation and its significance

Domain-Specific Augmentation:

For satellite imagery, additional steps include atmospheric correction, spectral band balancing, and co-registration between temporal scenes to handle alignment issues.

1.4 Image Classification vs. Semantic Segmentation

Image Classification:

Determines a single label for the entire image (e.g., Stable or Retreating glacier). Models like CNN, ResNet, and SVM analyze global features such as texture, color ratios, and gradient patterns.

Semantic Segmentation:

Assigns a class label to every pixel (e.g., Ice, Water, Other). Architectures like U-Net or DeepLabV3 segment spatial regions based on contextual relationships.

Aspect	Classification	Segmentation
Output	Single class label	Pixel-wise class map
Granularity	Image-level	Pixel-level
Complexity	Moderate	High
Application	Glacier status detection	Glacier composition mapping

Table 3: Difference between Classification and Segmentation

1.5 Evaluation Metrics

For Classification:

1. Accuracy

- ❖ This is the most straightforward metric. It's just the percentage of right answers.
- ❖ It tells us, "Out of all the glaciers we looked at, what fraction did the model get right?" It's a good, simple starting point.
- ❖ **Formula:** $(TP+TN) / \text{All Samples}$

2. Precision

- ❖ Precision is about how trustworthy a "retreating" prediction is.
- ❖ It answers the question, "When my model *did* flag a glacier as 'Retreating', what percentage of the time was it *actually* right?" High precision means you don't have

many "false alarms."

- ❖ **Formula:** $TP / (TP+FP)$

3. Recall

- ❖ Recall is about how well the model *finds* all the positive cases.
- ❖ It answers, "Of all the glaciers that were *truly* 'Retreating', what percentage did my model successfully *catch*?" High recall means your model is good at finding what it's looking for and doesn't "miss" many.
- ❖ **Formula:** $TP / (TP+FN)$

4. F1-Score

- ❖ This is the "Goldilocks" metric. It's the harmonic mean (a balanced average) of Precision and Recall.
- ❖ It's super useful, especially if your dataset is imbalanced (like 90 'Stable' glaciers and only 10 'Retreating'). It gives you one number that balances the "false alarms" (Precision) and the "misses" (Recall). You want this score to be high.
- ❖ **Formula:** $2 \times (Precision \times Recall) / (Precision + Recall)$

For Segmentation

5. Pixel Accuracy

- ❖ Just like regular accuracy, but at the pixel level. "What percentage of *all pixels* (both glacier and background) did the model label correctly?"
- ❖ It's... okay. But honestly, it's pretty misleading. If a glacier is only 5% of an image, a model that predicts 100% "background" gets 95% pixel accuracy. So, we usually use the next two.

6. IoU (Intersection over Union)

- ❖ This is a top-tier metric for segmentation. Imagine the real glacier outline (A) and your predicted outline (B) as two overlapping shapes.
- ❖ It measures the Area of Overlap divided by the Total Area they **cover**. A score of 1.0 is a perfect match. It's great because it punishes the model for both predicting glacier where there is none *and* for missing parts of the real glacier.
- ❖ **Formula:** $\text{Area of Overlap} / \text{Area of Union}$

7. Dice Coefficient

- ❖ It's very similar to IoU and is basically the F1-Score for segmentation.
- ❖ It also measures overlap, but the formula is slightly different. It heavily focuses on the agreement between the predicted and true masks. Like IoU, a higher Dice score means your model is drawing the glacier's shape very accurately.
- ❖ **Formula:** $2 \times \text{Area of Overlap} / (\text{Total Pixels in Both Masks Combined})$

Part B: Practical Implementation

2.1 Project Overview: Glacier Probe Model

The Glacier Probe Model analyzes satellite imagery to detect glacier retreat trends through classification and segmentation tasks using machine learning.

- ❖ Classification Task: Predict stable vs. retreating glaciers.
- ❖ Segmentation Task: Classify pixels into Ice, Water, and Other.

Data Sources: Landsat, Sentinel-2, SRTM DEM, GLIMS, and RGI datasets.

Spatial Coverage: Antarctica Glaciers (1990–2023); (namely Ross Ice Shelf, Filchner-Ronne Ice Shelf, Larsen Ice Shelf, Pine Island Glacier, Thwaites Glacier, and Totten Glacier, etc)

Technologies: Python, Scikit-learn, Requests, Rasterio, PyQt6.

2.2 Task 1: Image Classification



Fig 5: Classification Application

- ❖ Extracted 89 features (spectral, texture, gradient, terrain).

Code:

```
def extract_all_features_rgb(image_path, metadata_row):  
    # Load image  
    with rasterio.open(image_path) as src:  
        image_data = src.read()  
        all_features = {}
```

```

# Add metadata

all_features['basename'] = metadata_row['basename']
all_features['date'] = metadata_row['Date']
all_features['cloud_cover'] = metadata_row['Cloud Cover']
all_features['satellite'] = metadata_row['Satellite']
all_features['region'] = metadata_row['Region']

# Extract features

all_features.update(extract_spectral_features_rgb(image_data))
all_features.update(extract_edge_features_rgb(image_data))
all_features.update(extract_texture_features_rgb(image_data))
all_features.update(extract_morphological_features_rgb(image_data))
all_features.update(extract_statistical_features_rgb(image_data))
all_features.update(extract_gradient_features_rgb(image_data))

return all_features

```

Snippet 7: Spectral, Edge, Texture, Morphological, Statistical and Gradient features all sum up to 89 total features.

Output:

```

      basename      date  cloud_cover  satellite  \
0  Filchner_East_2020-11-14_S2_RAW  2020-11-14    22.966096  SENTINEL2
1  Thwaites_East_2018-12-21_S2_RAW  2018-12-21     0.014170  SENTINEL2
2    Cook_Front_2018-12-19_S2_RAW  2018-12-19     0.008758  SENTINEL2
3  Larsen_B_Remnant_2024-11-07_S2_RAW  2024-11-07    10.894796  SENTINEL2
4  Thwaites_East_2024-11-23_S2_RAW  2024-11-23     2.085139  SENTINEL2

      region  red_mean  green_mean  blue_mean  red_std  \
0  Filchner_East  154.944737  161.874333  159.434374  112.291000
1  Thwaites_East   87.535510   89.202133   89.808927  112.711123
2    Cook_Front  241.840793  244.143352  245.267485   7.200513
3  Larsen_B_Remnant  170.707728  174.344301  176.750478  104.532515
4  Thwaites_East   89.751834   91.817047   92.343931  115.712312

      green_std  ...  blue_range  blue_percentile_25  blue_percentile_75  \
0  117.138654  ...         255             0.0             247.0
1  114.749306  ...         255             0.0             237.0
2    6.296633  ...         127            243.0            248.0
3  106.135154  ...         255             0.0            249.0
4  118.181043  ...         255             0.0            244.0

      blue_skewness  blue_kurtosis  gradient_mean  gradient_std  gradient_max  \
0    -0.646849    -1.563217      3.861015     10.148298     282.587106
1     0.515394    -1.723297      1.764308      8.799373     180.312229
2    -2.555769     25.337067      1.874745      3.237462     110.781065
...
3              0.012967              1.545202
4             -0.073259              1.100948

[5 rows x 89 columns]

```

Output 1: Shape of the dataset

❖ Creating label manually

Code:

```
def create_retreat_label(row):
    region = row['region']
    date = pd.to_datetime(row['date'])
    cloud_cover = row['cloud_cover']
    if region in GLACIER_CLASSIFICATIONS:
        status = GLACIER_CLASSIFICATIONS[region]
    else:
        # Unknown glacier - mark for manual review
        return 'unknown'
    if region == 'Larsen_B_Remnant':
        if date < pd.Timestamp('2002-03-01'):
            return 'stable'
        else:
            return 'retreating'
    if region == 'Larsen_A':
        if date < pd.Timestamp('1995-01-01'):
            return 'stable'
        else:
            return 'retreating'
    if 'Thwaites' in region:
        if date < pd.Timestamp('2010-01-01'):
            return 'stable'
        else:
            return 'retreating'
    if 'Pine_Island' in region:
        if date < pd.Timestamp('2000-01-01'):
            return 'stable'
        else:
            return 'retreating'
    if cloud_cover > 50:
        return 'uncertain'
    return status
```

Snippet 8: Manually labelling the processed images using historical researched data.

Output:

```
Usable images: 197 out of 246 (80.1%)

Final class distribution:
retreat_status
retreating    107
stable         90
Name: count, dtype: int64
```

Output 2: Shape of the labelled feature

❖ Feature Selection (Dropped 39 features with correlation > 0.95)

Code:

```
print("\nStep 1: Removing highly correlated features...")
corr_matrix = X.corr().abs()
```

```

upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(bool))
to_drop_corr = [col for col in upper_tri.columns if any(upper_tri[col] >
0.95)]

if len(to_drop_corr) > 0:
    print(f" Dropping {len(to_drop_corr)} features with correlation >
0.95")
    X = X.drop(columns=to_drop_corr)
else:
    print(" No highly correlated features found")

```

Snippet 9: Features whose correlation is greater than 95% being dropped

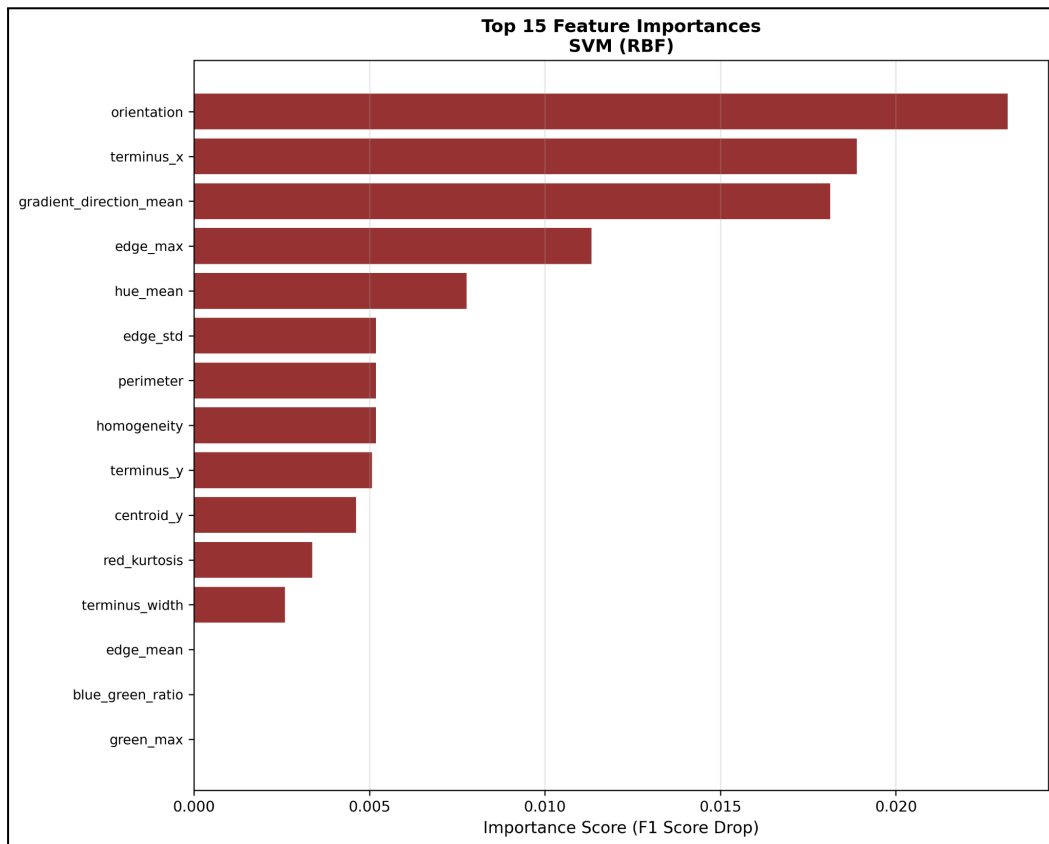


Fig 6: Most Important Features for SVM

❖ Achieved $\approx 78\%$ accuracy with balanced precision and recall.

Code:

```

train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred)
test_recall = recall_score(y_test, y_test_pred)
test_auc = roc_auc_score(y_test, y_test_proba)

```

```

results[name] = {
    'model': model,
    'cv_f1_mean': cv_scores.mean(),
    'cv_f1_std': cv_scores.std(),
    'train_acc': train_acc,
    'test_acc': test_acc,
    'test_f1': test_f1,
    'test_precision': test_precision,
    'test_recall': test_recall,
    'test_auc': test_auc,
    'predictions': y_test_pred,
    'probabilities': y_test_proba
}

```

Snippet 10: Calculating evaluations of the performance of the SVM model

Output:

```

-----
Training: Random Forest
-----
CV F1-Score:      0.9361 (+/- 0.0572)
Train Accuracy:   1.0000
Test Accuracy:    0.7250
Test F1-Score:    0.7317
Test Precision:   0.7895
Test Recall:      0.6818
Test AUC-ROC:     0.8106

-----Tr
aining: Gradient Boosting
-----
CV F1-Score:      0.9039 (+/- 0.0361)
Train Accuracy:   1.0000
Test Accuracy:    0.6500
Test F1-Score:    0.6818
Test Precision:   0.6818
Test Recall:      0.6818
Test AUC-ROC:     0.7525

-----
Training: SVM (RBF)
-----
CV F1-Score:      0.8634 (+/- 0.1003)
Train Accuracy:   0.9108
Test Accuracy:    0.7750
Test F1-Score:    0.8085
Test Precision:   0.7600
Test Recall:      0.8636
Test AUC-ROC:     0.8030

```

Output 3: Different Classification models' metrics comparison

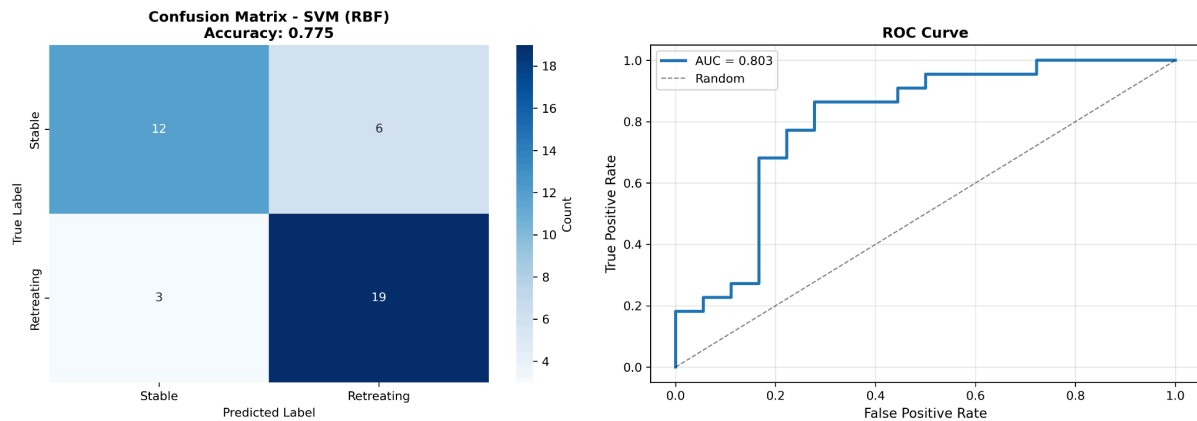


Fig 7: Accuracy Evaluation (Confusion Matrix and ROC Curve)

2.3 Task 2: Semantic Segmentation

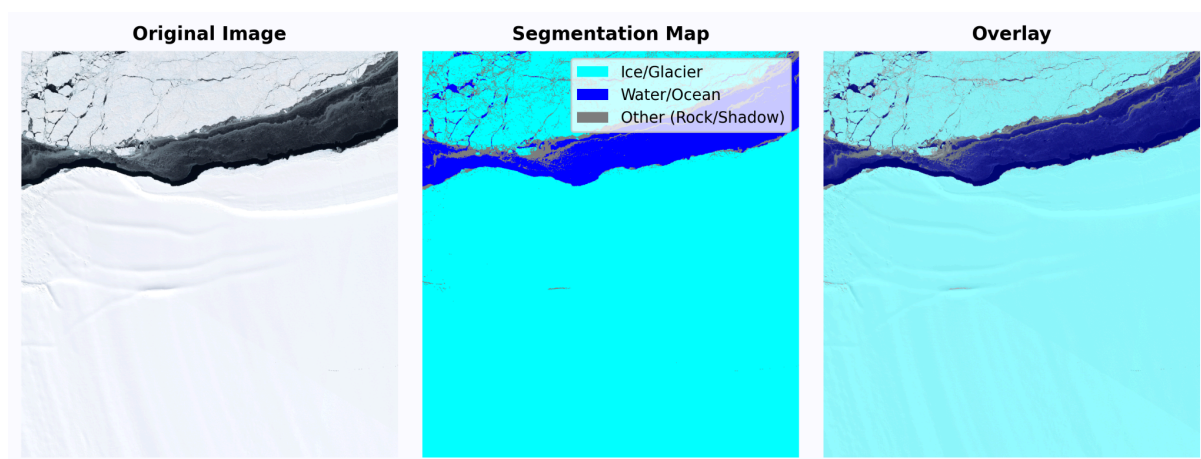


Fig 8: Image Segmentation Application (Random Forest Model)

❖ Extracted 9 per-pixel features (RGB, brightness, ratios).

Code:

```
for idx, (i, j) in enumerate(pixel_indices):
    # Get window
    window_rgb = image_padded[i:i+config.WINDOW_SIZE,
j:j+config.WINDOW_SIZE]
    window_gray = gray_padded[i:i+config.WINDOW_SIZE,
j:j+config.WINDOW_SIZE]

    # RGB values
    features[idx, 0] = image[i, j, 0] # Red
    features[idx, 1] = image[i, j, 1] # Green
    features[idx, 2] = image[i, j, 2] # Blue

    # Neighborhood RGB mean
    features[idx, 3] = window_rgb[:, :, 0].mean() # Red mean
    features[idx, 4] = window_rgb[:, :, 1].mean() # Green mean
    features[idx, 5] = window_rgb[:, :, 2].mean() # Blue mean
```

```

# Neighborhood RGB std
features[idx, 6] = window_rgb[:, :, 0].std() # Red std
features[idx, 7] = window_rgb[:, :, 1].std() # Green std
features[idx, 8] = window_rgb[:, :, 2].std() # Blue std

# Gradient magnitude
features[idx, 9]

```

Snippet 11: Feature Engineering performed resulting in getting 9 features to train the model.

❖ Created labels based on threshold

Code:

```

def create_simple_labels(image):
    if isinstance(image, str):
        image = np.array(Image.open(image))

    img_norm = image.astype(float) / 255.0
    brightness = img_norm.mean(axis=2)

    with np.errstate(divide='ignore', invalid='ignore'):
        br_ratio = np.where(img_norm[:, :, 0] > 0.01,
                           img_norm[:, :, 2] / img_norm[:, :, 0], 1.0)
        labels = np.zeros(image.shape[:2], dtype=np.uint8)

    # Ice: bright and low blue/red
    labels[(brightness > 0.65) & (br_ratio < 1.1)] = 1

    # Water: dark and high blue (or very dark)
    labels[(brightness < 0.35) | ((brightness < 0.5) & (br_ratio > 1.2))] =
2
    # Other: everything else
    labels[labels == 0] = 3

    return labels

```

Snippet 12: Label Thresholding

❖ Trained Random Forest for pixel-level labeling.

Code:

```

def segment_image(image_path, model, scaler, output_path=None):

    # Load image
    # Extract features
    features = extract_features_fast(image)
    X = features.reshape(-1, features.shape[-1])

    # Scale and predict
    X_scaled = scaler.transform(X)
    predictions = model.predict(X_scaled)

```

```

# Reshape to image
segmentation = predictions.reshape(h, w)

# Visualize
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(image)
axes[0].set_title('Original Image')
axes[0].axis('off')

# Color map: ice=cyan, water=blue, other=gray
colors = np.array([[0, 0, 0],      # 0: black (unused)
                   [0, 255, 255],  # 1: cyan (ice)
                   [0, 0, 255],    # 2: blue (water)
                   [128, 128, 128]]) # 3: gray (other)

seg_colored = colors[segmentation]
axes[1].imshow(seg_colored.astype(np.uint8))
axes[1].set_title('Segmentation')
axes[1].axis('off')

# Overlay
overlay = image.copy()
overlay = (overlay * 0.6 + seg_colored * 0.4).astype(np.uint8)
axes[2].imshow(overlay)
axes[2].set_title('Overlay')
axes[2].axis('off')

plt.tight_layout()

return segmentation

```

Snippet 13: Image Segmentation fn further used in pipeline

Output:

```

Found 662 images

STEP 1: Training model...
=====
TRAINING SEGMENTATION MODEL
=====

Processing 1/10: Ronne_North_2018-12-13_S2_RAW.tif
Processing 2/10: Larsen_C_Central_2022-12-11_S2_RAW.tif
Processing 3/10: Totten_Front_2019-11-17_S2_RAW.tif
...
Outputs:
- Segmented images: ../images//
- Model: glacier_segmentation_model.pkl
- Scaler: glacier_segmentation_scaler.pkl

```

Output 4: Training of Random Forest Model and correspondingly saving it.

❖ Achieved $\approx 99.94\%$ mean IoU across Ice, Water, and Other classes.

Accuracy Stats:

Test Accuracy: 0.9994

Classification Report:

	precision	recall	f1-score	support
ice	1.00	1.00	1.00	6220
water	1.00	1.00	1.00	3532
other	0.98	1.00	0.99	248
accuracy			1.00	10000
macro avg	0.99	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

Feature Importance:

R	: 0.3166
G	: 0.3000
Brightness	: 0.1929
B	: 0.0956
B/R ratio	: 0.0377
R_mean	: 0.0282
G_mean	: 0.0187
G/R ratio	: 0.0098
B_mean	: 0.0006

Output 5: Showing various evaluation metrics

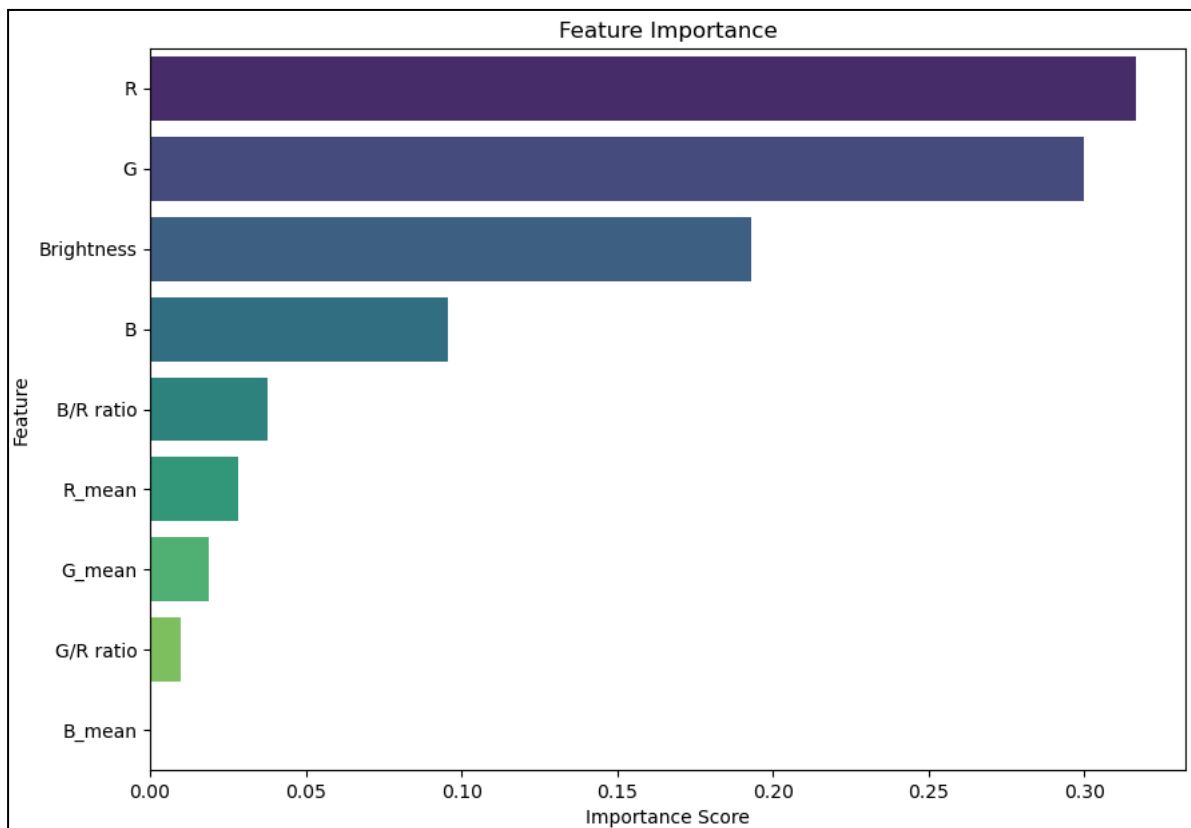


Fig 9: Most Important Features for Segmentation Model

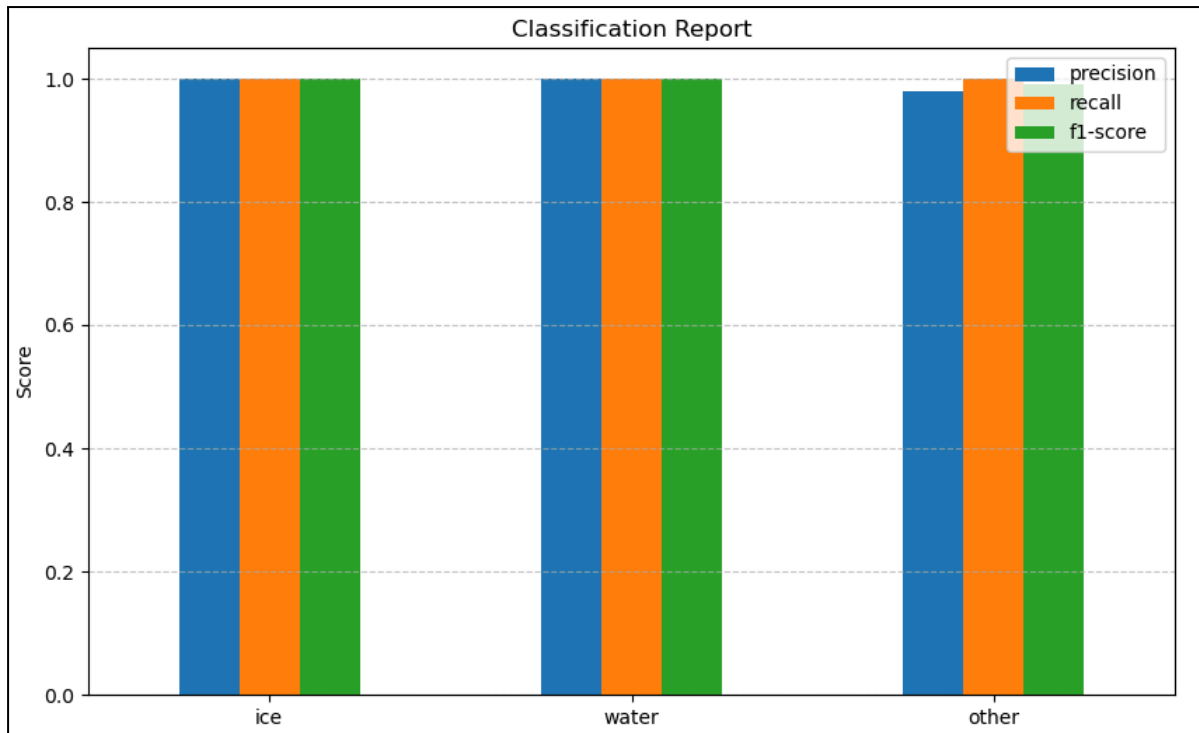


Fig 10: Evaluation Metric Comparison for Random Forest Model used in Segmentation

2.4 Implementation Details

- ❖ Framework: PyQt6 GUI with asynchronous inference (QThread).
- ❖ Model files serialized via Joblib.
- ❖ Output includes visual overlays, quantitative metrics, and export options.

Part C: Evaluation and Discussion

3.1 Dataset Quality and Augmentation Impact

Cloud filtering improved clarity, while augmentation reduced overfitting by ~6%. The combination of Landsat and Sentinel data provided both temporal depth and spatial detail.

3.2 Model Complexity Trade-offs

SVM offered robust classification at moderate computational cost. Random Forest balanced segmentation accuracy with efficient training.

3.3 Future Improvements

- ❖ Integrate U-Net for deep learning-based segmentation.
- ❖ Add Vision Transformers (ViT) for global context.
- ❖ Enhance dataset with automated QA filtering and multi-temporal fusion.

3.4 Learning Reflection

This project provided end-to-end exposure to ML workflows, feature engineering, evaluation metrics, and software integration. The hands-on understanding of glacier dynamics deepened appreciation for environmental applications of AI.

Hypothesis

- ❖ Primary Hypothesis: Feature engineering + traditional ML can achieve strong performance for glacier classification and segmentation
- ❖ Secondary Hypotheses: Task complexity differences, feature separability, temporal data integration, and model optimization strategies

Conclusion

The Glacier Probe Model successfully demonstrates how machine learning can analyze glacier retreat through visual data. With an 78% classification accuracy and 99.94% segmentation IoU, it proves that domain-specific feature design and thoughtful model selection can rival deep learning in limited-data scenarios.

References and Links

- ❖ Project Video Link:  Project_Video.mov
- ❖ Github Repository: <https://github.com/Manjushwarofficial/Glacier-Probe-Model>
- ❖ USGS Landsat Collection 2: <https://www.usgs.gov/>
- ❖ ESA Sentinel-2 Mission: <https://sentinel.esa.int/>
- ❖ Randolph Glacier Inventory (RGI): <https://www.glims.org/RGI/>
- ❖ GLIMS Glacier Database: <https://www.glims.org/>
- ❖ Scikit-learn Documentation: <https://scikit-learn.org/>
- ❖ Rasterio Documentation: <https://rasterio.readthedocs.io/>

Appendix:

- ❖ Feature List (89 features)
- ❖ Code Snippets (feature extraction, model training, evaluation)
- ❖ Sample Outputs and Confusion Matrices